assignment4_part1

October 24, 2026

1 Assignment 4: Dealing with Sequences Real-World Data (Part I)

Welcome to Assignment 4, the last assignment in this course. In this assignment, we will explore the sequence representation of data. Lots of real-world data can be represented as sequences. Among them, text data is typical and widely available, which we will use for this assignment. We will look at the Tweets with the colorful emojis again, but this time we are not going to filter the textual content. Several toolkits or packages have been developed to process text data. In this assignment, we will rely on the NLTK package (Natual Language Toolkit).

In this assignment, you will: * Tokenize text data and extract ngrams and skipngrams. * Implement the calculation of edit distance. * Find near-duplicate sequences of a given piece of text.

First, let's load the dependencies and the Tweets.

[1]:
```
import pandas as pd
import csv

import nltk

from collections import Counter
```

[2]:
```
tweets = []
with open('assets/tweets.txt', encoding='utf-8') as f:
    for line in f:
        if len(line) > 0:
            tweets.append(line.strip())
tweets[:5]
```

[2]:
```
["RT @stunnaboyco: @LUSCIOUSLOUIS that's lunchtime    rite there
https://t.co/m0Wngznctw",
 '@RossoRestaurant looking forward to joining you for lunch tomorrow to
celebrate my husbands birthday!  ',
 'Out wth my lilbro 4 ice-cream   https://t.co/dFEimQiF3n',
 'Kuku  le Dickson  tsa ba bangwe di behaviour okare Clientelle funeral cover
They can cover up to 15 guys/ladies. #TheLivingSoul ',
 'RT @Thabang92416252: "@PinexAndApplex:   @EmteeSA - We up
https://t.co/nGbvFW2CUA"YEE']
```

To construct the sequence representation of the Tweet, we need to *tokenize* each Tweet into a sequence of language units, which in our case are words. For this assignment, we will use the TweetTokenizer API. For some languages, however, tokenization can be challenging.

[3]:
```
tokenizer = nltk.tokenize.casual.TweetTokenizer()
```

[4]:
```
tokenizer.tokenize(tweets[0])
```

[4]:
```
['RT',
 '@stunnaboyco',
 ':',
 '@LUSCIOUSLOUIS',
 "that's",
 'lunchtime',
 ' ',
 ' ',
 ' ',
 ' ',
 'rite',
 'there',
 ' ',
 ' ',
 ' ',
 ' ',
 ' ',
 ' ',
 ' ',
 ' ',
 ' ',
 'https://t.co/m0Wngznctw']
```

For a quick sanity check, we can calculate the word frequency and see which ones are the most frequently used. With the `Counter` object, we can easily obtain the most frequently used words and their numbers of occurrences.

[5]:
```
unigram_counter = Counter()
for tweet in tweets:
    unigram_list = tokenizer.tokenize(tweet)
    unigram_counter.update(unigram_list)
unigram_counter.most_common(20)
```

[5]:
```
[('!', 4748),
 (':', 4601),
 ('RT', 3757),
 ('', 2742),
 ('…', 2713),
 ('.', 2413),
 (' ', 2135),
```

```
 (',', 2084),
 ('a', 1830),
 ('to', 1819),
 ('the', 1806),
 ('you', 1670),
 ('and', 1646),
 (' ', 1413),
 ('I', 1403),
 (' ', 1294),
 (' ', 1274),
 ('Happy', 1269),
 (' ', 1226),
 (' ', 1183)]
```

One common type of defined sequential patterns are n-grams. Particularly, 1-grams are often called "unigrams", 2-grams called bigrams, and 3-grams trigrams. Let's examine the bigrams of a Tweet:

[6]:
```
bigram_list = list(nltk.bigrams(tokenizer.tokenize(tweets[0])))
bigram_list
```

[6]:
```
[('RT', '@stunnaboyco'),
 ('@stunnaboyco', ':'),
 (':', '@LUSCIOUSLOUIS'),
 ('@LUSCIOUSLOUIS', "that's"),
 ("that's", 'lunchtime'),
 ('lunchtime', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', 'rite'),
 ('rite', 'there'),
 ('there', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', ' '),
 (' ', 'https://t.co/m0Wngznctw')]
```

Note that each bigram is represented as a tuple of two strings.

1.0.1 Exercise 1. Find the most frequent bigrams (20 pts)

Please complete the `freq_bigram` function to find the n most frequent bigrams. Your function should return a list of `top_n` tuples. Each of the tuples should contain a bigram tuple (such as (' ',

‘ ’)) and its number of occurrence.

Hint: The code above, as in previous questions, shows you how to accomplish this.

[28]:
```
def freq_bigrams(tweets, top_n):
    bigram_counter = Counter()
    for tweet in tweets:
        # YOUR CODE HERE
        #raise NotImplementedError()
        bigrams = list(nltk.bigrams(tokenizer.tokenize(tweet)))
        bigram_counter.update(bigrams)
        #words = tweet.lower().split()
        #print(tweet, words)
        #for i in range(len(words)-1):
        #    bigram_counter[(words[i], words[i+1])] += 1
    return bigram_counter.most_common(top_n)
```

[29]:
```
freq_bigrams(tweets, 10)
```

[29]:
```
[(('!', '!'), 1334),
 ((' ', ''), 600),
 (('Happy', 'Birthday'), 570),
 ((’', 's'), 483),
 ((' ', ''), 437),
 (('Happy', 'birthday'), 347),
 ((' ', ' '), 288),
 ((' ', ' '), 277),
 ((' ', ' '), 276),
 ((' ', ' '), 264)]
```

[30]:
```
# This code block tests whether the `freq_bigrams` function is implemented␣
 ↪correctly.
# We hide some tests. Passing the displayed assertions does not guarantee full␣
 ↪points.
answer = freq_bigrams(tweets, 10)
assert answer[0] == (('!', '!'), 1334)
assert answer[8] == ((' ', ' '), 276)

answer2 = freq_bigrams(tweets[:5000], 5)
assert answer2[0] == (('!', '!'), 682)
assert answer2[2] == (('Happy', 'Birthday'), 290)
```

Similarly, you can generate trigrams by calling the `nltk.trigrams` API.

1.0.2 Exercise 2. Find the most frequent skipgrams (20 pts)

In this exercise we will compute another commonly defined type of sequential patterns – the skip-grams. Luckily this is also supported by NLTK. You can find the documentation here.

Please implement the `freq_skipgrams` function to calculate the most frequently used k-skip-n-grams. Your function should return a list of `top_n` tuples. Each of the tuples should contain a k-skip-n-gram tuple (such as ('Happy', 'Birthday', ' ')) and its number of occurrences.

Hint: This code will be very similar to the code for the previous question. You must simply adjust the arguments for use with the nltk.skipgrams function.

[32]:
```
def freq_skipgrams(tweets, n, k, top_n):
    skipgram_counter = Counter()
    # YOUR CODE HERE
    for tweet in tweets:
        # YOUR CODE HERE
        #raise NotImplementedError()
        skipgrams = list(nltk.skipgrams(tokenizer.tokenize(tweet), n, k))
        skipgram_counter.update(skipgrams)
        #words = tweet.lower().split()
        #print(tweet, words)
        #for i in range(len(words)-1):
        #    bigram_counter[(words[i], words[i+1])] += 1
    #raise NotImplementedError()
    return skipgram_counter.most_common(top_n)
```

With this function, you can find the 10 most frequent 2-skip-trigram with the following command.

[33]:
```
freq_skipgrams(tweets, n=3, k=2, top_n=10)
```

[33]:
```
[(('!', '!', '!'), 511),
 (('', ' ', ''), 393),
 ((' ', '', ' '), 365),
 ((' ', ' ', ' '), 282),
 (('Happy', 'Birthday', '!'), 260),
 (('!', '!', ' '), 239),
 (('', '', ''), 222),
 (('RT', ':', 'Happy'), 212),
 ((' ', '', ''), 182),
 (('Birthday', '!', '!'), 161)]
```

[34]:
```
# test
# This test cell contains hidden tests. Passing the displayed assertions does␣
 ↪not guarantee full points.
answer = freq_skipgrams(tweets, n=3, k=2, top_n=10)
assert answer[0] == (('!', '!', '!'), 511)
assert answer[3] == ((' ', ' ', ' '), 282)
```

Ngrams and skipgrams are commonly used in text mining, biological sequence mining, and behavior mining tasks. They are directly used as basis for tasks like phrase detection, named-entity detection, and motif detection, and they are also used as features for building machine learning models in general. Have fun using them in your own data analysis!